知乎 DataClaw

从 MetricFlow 到 AI 原生语义层的演进实践

汤晋瑄-知乎-大数据部

目录

CONTENT

01 业务背景

02 架构演进

03 落地细节

04 落地效果

05 总结展望

01 业务背景

业务痛点：80% 的时间花在体力劳动上

① 取数依赖分析师

业务方等待时间长

需求排队，口径来回确认

等待久

② SQL 编写成本高

技术门槛限制自助取数

非技术同学拿不到数

门槛高

③ 指标口径不一致

分析结果存在差异

同一指标两套答案

口径乱

大模型强得可怕，但它不了解业务知识和指标口径

02 架构演进

第一阶段：基于 dbt MetricFlow 的语义层

技术架构

dbt Semantic Layer → MetricFlow 引擎 → BM25 指标检索

核心特点：SQL 完全由 MetricFlow 引擎生成，准确性有保障

优势

- 规范化
- 类型安全
- SQL准确

局限性

- MetricFlow 复杂，特殊指标支持成本高
- 不够灵活，无法充分发挥 LLM 的能力
- 维护 MetricFlow 配置成本高

MetricFlow 保证了 SQL 的准确性，也限制了 AI 的想象力。

第二阶段：AI 原生知识库

设计原则：让 AI 自动维护和使用数据资产

指标定义由 LLM 生成

人工确认后进入知识库

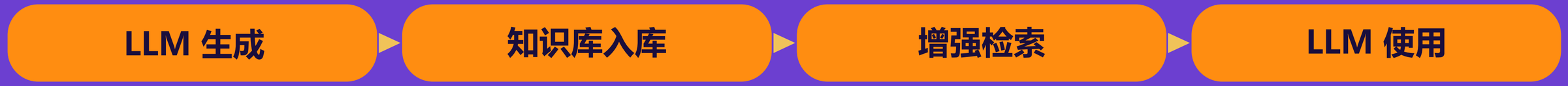
知识库是语义层核心

AI 原生语义层的支撑底座

闭环自运转

生成 → 入库 → 检索 → 使用

完整闭环



面临的问题：多跳关系检索效果有限

例如「用户 → 订单 → 商品」三跳关系，靠知识库检索答不上来。

与其让 AI 遵守人的规则，不如让 AI 学会自己生产规则。

第三阶段：轻量级本体图谱建设

设计原则：让 AI 抽取本体图谱，最终由专家确认

本体图谱的三层价值

L1 · 连接

知道「和谁有关」

· 用户 → 观看 → 内容，多跳可达

· 解决跨表、跨实体「找不到」

· 仅连接层：有关系，不解释为什么

→

L2 · 理解

懂得「业务是什么」

· 概念、属性、规则一起入图

· 口径、字段归属、规则都可查

· Agent 直接读到语义，不用猜

→

L3 · 推理

推出「结果会怎样」

· 沿 isa / about / 规则约束推导

· 口径自动展开到事实、维度、过滤条件

· 从「查数」到「下结论、讲原因」

知识库回答「是什么」；本体图谱靠 L2/L3 真正回答「为什么、会怎样」—— 关系只是起点，不是终点。

三阶段演进

① MetricFlow 语义层 · SQL 准确但不灵活 → ② 知识库建设 · AI 生产与消费数据资产 → ③ 本体图谱 · 懂业务、能推理

图谱的价值不在关系本身，而在关系之上长出来的业务判断。

03 落地细节

架构总览


大数据 AI 平台 / Zwork 统一工作台 / Codex / Cursor / Workbuddy
一站式入口 · 任务调度 · 对话交互 · 结果展示
大数据 CLI
安全管控 · 任务分发
AI 专家
任务承接
数据分析 Agent
数据分析 · 报表生成 · 洞察
AB 实验 Agent
实验设计 · 数据分析 · 结论
画像 Agent
用户画像 · 标签体系 · 画像洞察
埋点 Agent
埋点分析 · 事件追踪 · 效果评估
有资源 · 数据能力供给
SQL 查询服务
计算引擎
存储引擎
懂业务 · 业务知识理解与映射
知识检索服务
知识库
本体图谱
业务逻辑映射
产品功能映射
会干活 · 能力复用与任务执行
数据 SKILL 市场
表元数据、数仓代码、血缘关系
埋点元数据、产品 PRD
数据平台
数据来源
数据仓库 + 数据产品
业务数据
业务产品 + 数据产品
业务应用
数仓 + 数据产品+数据分析+业务同学

知识库 · 混合检索



全文管准，向量管懂，WRRF 管谁排在前面。

知识库 · WRRF 融合细节

不看分数看排名：两路 rank 各取 1/(k+rank)，加权求和后重排

WRRF_score = vectorWeight × 1/(k + vector_rank) + fulltextWeight × 1/(k + fulltext_rank)

只在一路出现的 chunk 只累加那一项；两路都召回到的 chunk 两项都加

召回与融合流程

向量路

1 − COSINE_DISTANCE(查询向量, embedding) 降序取前 N → 得 rank

全文路

MATCH(...) AGAINST(... IN NATURAL LANGUAGE MODE) 降序取前 N → 得 rank

类别权重钩子

进入 RRF 前：分数 × 类别权重 → 重排 → 重赋 rank（影响排名，不做分数后处理）

calculateWRRF

两路 rank 代入公式累加 → 按 WRRF 分数降序取 top-K

参数与配置

参数	默认值	环境变量
vectorWeight	1.0	WRRF_VECTOR_WEIGHT
fulltextWeight	1.0	WRRF_FULLTEXT_WEIGHT
k	60	WRRF_K
perPathTopK	20	WRRF_PER_PATH_TOPK

k=60 是标准 RRF 经验值：抹平头部排名分差，让低排名结果也有贡献空间

融合的是排名不是分数：谁被两路同时认可，谁就排得靠前。

知识库 · 定制化切片

知识类型	切片粒度	切片内容
指标	一指标一切片	指标名称、描述、完整口径、物理表映射
表元数据	表级 + 字段级	表级：表名、描述、业务域 字段级：字段名、字段类型、业务描述
SQL 模板	按语句切分	SQL 语句、CTE 语句分别成片
业务知识	通用 Markdown 切片	通用逻辑兜底，兼容未知类型

切片不是切得越碎越好，是按知识的形状下刀。

本体图谱 · 轻量化设计

逻辑层定义「是什么」，物理层只锚「去哪查」

逻辑层 • 本体（概念 / 属性 / 规则）

实体 • Entity — OWL Class
业务对象：用户、内容 → dim_* 维度表

事实 • Fact — OWL Class
行为事件：观看、付费 → dwd_* / dws_*

值概念 • ValueConcept — OWL Class
受控词表：性别、平台等枚举

属性 • Attribute — OWL DataProperty
数据属性：字段名 / 类型 / 取值，实体与事实共用

规则 • Rule — OWL Rule / SWRL
业务规则：条件分支、口径约束、衍生属性

物理层 • 锚点（每个节点自带）

表锚点 • table_ref / table_refs
实体、事实各自挂表：dim_* / dwd_*，多表按 priority

字段映射 • column_maps
事实用 measures + dimensions，度量与维度落到物理列

取值与口径 • value_source / metrics
枚举取值归值概念；指标口径挂在事实上

属性锚点 • attribute.column_maps
实体用 attributes；column → columns → column_maps

规则锚点 • rule.inputs / cases
规则声明输入列与条件分支，按属性路由不同列

本体论映射： 概念 → owl:Class · 属性 → owl:DatatypeProperty / owl:ObjectProperty · 规则 → SWRL Rule（业务规则）

关系层 • ObjectProperty isa 继承 · about 关联实体 · has_dimension 关联维度 · 自定义关系（遵循 OWL，自由定义）

示例链路（真实字段）

User(dim_user) ──has_dimension(member_id)──▶ 观看事实(dwd_user_view_content) ──about──▶ Content(dim_content)

事实自带表与度量：table_ref = dwd_user_view_content；measures.viewDuration → column: view_duration；指标口径 SUM(view_duration) 同样挂在事实上

本体图谱 · 轻量级技术架构

存储用关系数据库表，能力统一收口到 MCP

存储

核心数据用两张表承接
nodes + edges

完整定义存 JSON
节点/边的定义整体放 properties 字段

版本是 YAML 快照
快照存 BOS，DB 只留版本元数据

计算

递归 CTE 多跳
WITH RECURSIVE 逐跳展开节点与边

邻居 1–4 跳
默认 1 跳，支持 in/out/both 与关系过滤

路径默认 4 跳
上限 6 跳，超出返回未找到并给提示

接口

MCP 统一出口
只读 7 个 kg_* + 编辑 5 个 kge_*

双视图返回
agent 视图瘦身给 LLM，full 视图给人看

自带下一步提示
返回体带 next_hint 引导下一步查询

轻量化的边界

只维护节点自带的锚点，不做全量物理建模；查询侧用 Schema 匹配兜底

图计算限定在 4 跳内（路径上限 6 跳），换取可预期的查询耗时

本体图谱 · 管理维护

LLM 编辑、人类审核：图谱可看、版本可溯、发布可控

拉取版本
kge_pull → 拉到本地

→

LLM 编辑
修改：加实体、补列映射

→

推送预览
kge_push 生成预览版本

→

差异对比
可视化对比

→

人工确认发布
审核通过 → 发布为正式版本

版本管理

三段版本号
正式 X.Y | 工作线 X.(Y+1) | 预览 X.(Y+1).N

快照归档可回滚
版本 YAML 存 BOS，DB 存元数据，可回溯

状态机清晰
正式版 published/archived；预览 draft/published/abandoned

质量与审核

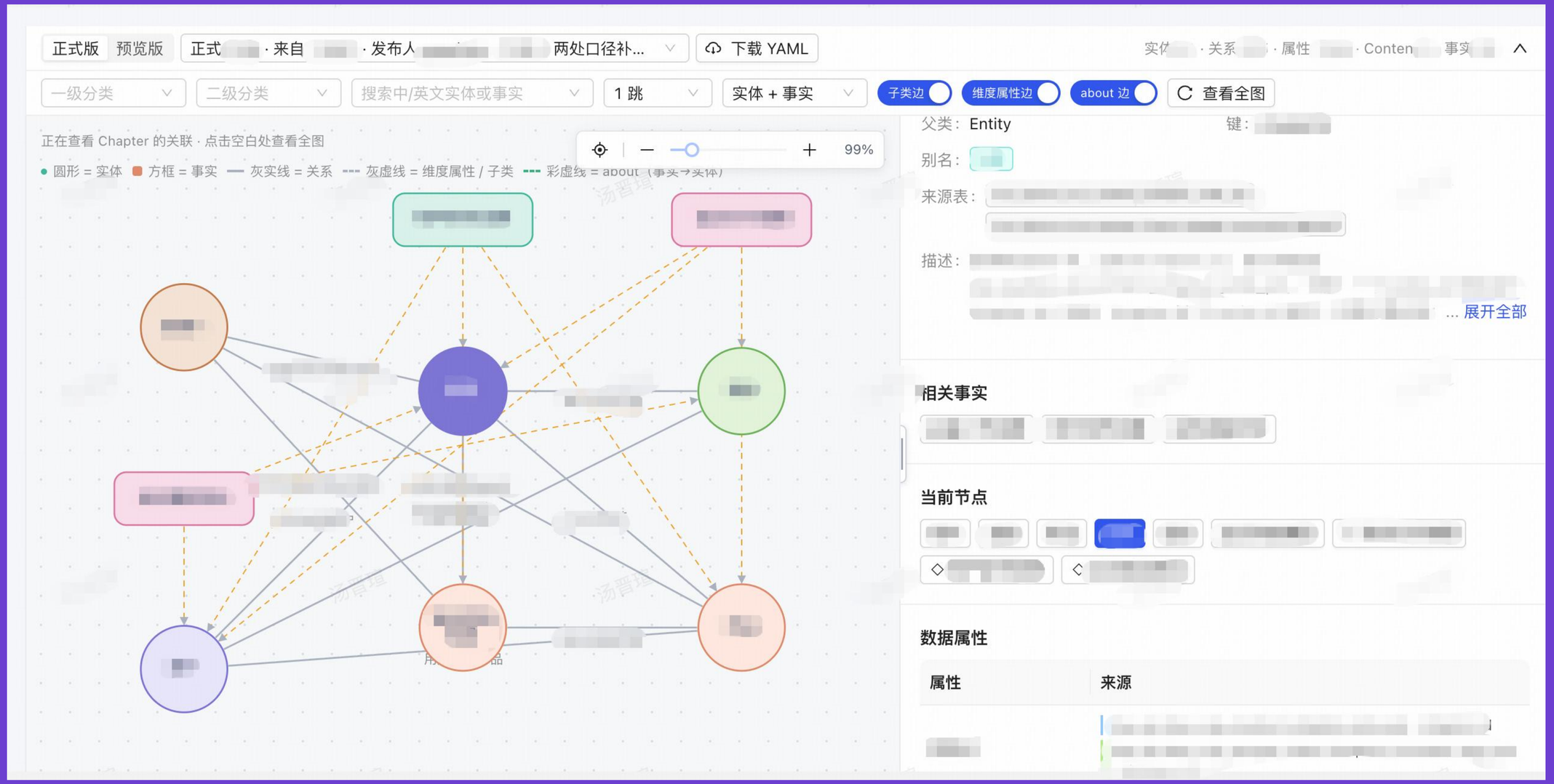
图谱校验
kg_validate + 导入校验：缺表锚点、规则错误

人工把关
LLM 只产出预览，发布权在人类审核

可视化审核
图谱结构图 + 版本/预览详情，发布确认与回滚在同处完成

版本即契约：Agent 只读正式版本，所有编辑先落预览，避免误改线上图谱

本体图谱 · 管理界面


正式版
预览版
下载 YAML
一级分类
二级分类
搜索中/英文实体或事实
1 跳
实体 + 事实
子类边
维度属性边
about 边
查看全图
正在查看 Chapter 的关联 · 点击空白处查看全图
99%
圆形 = 实体
方框 = 事实
灰实线 = 关系
灰虚线 = 维度属性 / 子类
父类：Entity
键：
别名：
来源表：
描述：
... 展开全部
相关事实
当前节点
数据属性
属性
来源

落地挑战一：四类知识共用一套切片

起因

四类知识初期共用一套切片 + 单一向量检索，先求上线。

踩坑

噪音最大的是表元数据 —— 问「DAU 从哪张表取」，回来几十个同名字段，答非所问。

定位

表结构被按句子切碎了；核心宽表、认证表同权重，业务优先级没进检索。

改法

① 按知识类型定制切片，通用逻辑兜底 ② 表元数据分层：先表级、再字段级 ③ 表按照业务优先级增加权重

上线那天是最便宜的一天，真正贵的是之后每天的 bad case。

落地挑战二：我们差点上了图数据库

起因

多跳问题在纯向量检索下答不上来，第一反应是上重量级图数据库。

经过

原型验证后发现：真实查询集中在 1-3 跳，

图数据库的运维成本远超收益，最终方案被否掉。

改法

① LLM 抽取实体与关系，人工审核后入库

实体：指标、表、字段、业务概念

关系：指标依赖、表关联、字段引用、业务归属

② 轻量图存储 + AI 友好的自然语言查询接口

③ 只承诺支持 1-3 跳查询

图谱结果与知识库检索融合

沉淀

判据变了 —— 不再问「哪个技术更先进」，而是问「我们的问题有多大」。

技术选型最贵的坑，是用先进性做判据，而不是用问题规模做判据。

04 落地效果

量化效果：规模是投入，效果才是收益

规模数据（说明覆盖面，不代表效果）

指标口径沉淀为知识资产
千级

SQL 沉淀为可复用知识资产
万级

用户渗透率
0 → 65%+

效果数据

① 取数依赖分析师、等待久	等待时长从天级降到分钟级
② SQL 编写门槛高	非技术同学简单自助完成取数占比 100%
③ 指标口径不一致	统一走知识库口径后，口径类答疑/返工下降 50%

规模数据说明投入，效果数据才是收益。

真实测评 · 有问题，就会有答案

题集来自真实提问，答案由专家标注，判定由 AI 辅助完成

整体准确率

95%

题集 = 核心指标改写问题 + 业务用户真实问题
两类题集合并计算

业务用户真实问题

知识库阶段

70%

→ 提升 11pp

本体图谱 + 知识库阶段

92%

测评怎么做的

题集来源
业务真实提问埋点数据

答案标注
专家团队标注正确答案

判定方式
AI 辅助判断（意图匹配 + SQL 等价）

评测怎么落地的

评测链路已脚本化：真实问题提取 → 专家标注用例 → LLM 判定 → 报告归档
同一套用例可重复回归，结论可复核

效果展示


有新版 Chrome 可用
Bookmarks
首页 | Golang Chin...
配置vim的Go开发...
maven
wechat-developm...
cloud-service
learning
Imported From Fir...
操作系统
人工智能-机器学习
所有书签
大数据平台
搜索指标、维度、表、报表等
搜 索
汤晋瑄
AI
工具
分析
开发
今天，我们一起完成什么数据任务?
选择一个专业场景，或者直接告诉我你的需求。
数据分析专家
数据查询
指标口径
归因分析
AB 实验分析专家
AB 实验
显著性检验
指标归因
用户画像专家
人群圈选
人群洞察
Lookalike
埋点设计专家
需求理解
事件设计
方案输出
看板专家
描述你的问题，输入 @ 选择专家，或上传数据文件开始分析...

一位业务同学的一天

Before

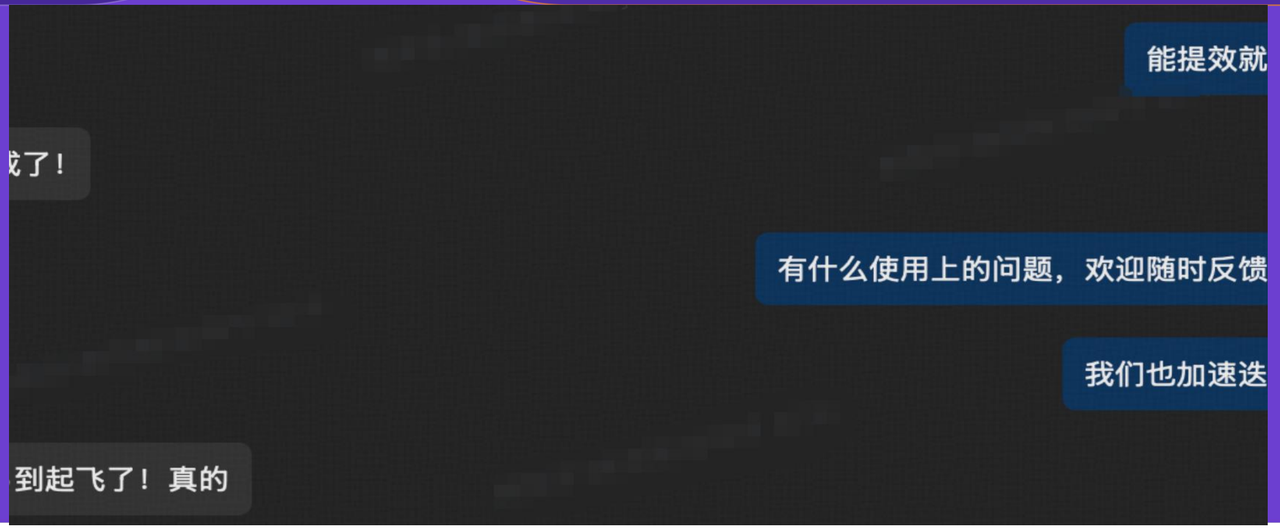
想看一个数，先排队

- 提需求、写背景说明
- 排进分析师队列
- 等待天级才拿到结果
- 口径还要来回确认一轮
- 赶上临时汇报基本来不及

After

自己问一句，直接进决策会

- **自己问一句「上周 DAU 为什么涨了」**
- **分钟级拿到可复用 SQL 和结论**
- **直接进决策会**



最贵的从来不是技术，是等待时间。

05 总结与展望

三条方法论

01

定制化切片

切片按知识的形状

回去怎么用

先盘点你的知识类型，别用一套切片打天下。

02

混合检索

检索按业务的优先级

回去怎么用

给你的表资产分权重，核心宽表、认证表先进知识库。

03

AI 原生

图谱按问题的规模

回去怎么用

先量化你的查询跳数，再决定要不要上图谱。

切片按知识的形状，检索按业务的优先级，图谱按问题的规模 —— 别让通用方案替你做判断。

关于CLI的实例——ThinkingAI CLI 的业务调用

示例任务：核对最近一周的活跃变化

内容	在一次分析任务中的作用
ae-cli	在授权范围内发现并调用企业已有能力
Skill	组织分析步骤，明确结论需要的证据
业务知识与口径	说明对象、统计口径、资产选择与例外
数据和分析平台	提供可复算的查询、分析结果与数据依据

先确认去重对象、统计时区与排除规则，再解释变化

公开入口：github.com/ThinkingAIAgenticEngine/ae-cli

避坑清单

① 知识建设，人工参与的比例有多大？

→ 人工确认该放在流程的哪个位置 —— 前移是成本，后移是事故。

② 切片策略，是四类知识共用一套，还是按类型定制？

→ 共用一套，会有比较大大噪音

③ 技术方案，是按问题规模选的，还是按技术先进性选的？

→ 我们真实查询集中在 1-3 跳。

这三条不是知识点，是我们交过的学费。

从「辅助人工构建语义」到「全自动构建知识飞轮」

从人工指挥构建语义到 AI 自主构建可靠知识

用户问题

检索本体图谱、知识库

判断知识是否缺失

如果不缺失，回答用户问题
如果缺失，进入知识自动构建

知识构建

自动构建知识并回测
人工审核后正式入本体图谱、知识库

未来方向：让 本体图谱、知识库 主动发现知识缺失，并构建新知识

AI 原生不是技术选择，是设计理念。

THANKS

演讲嘉宾：汤晋瑄